



▶ **JAVA DEMO**

Hello World Program

The Java program:

- Prints CLI arguments.
- Prints environment variables.
- Prints the content of files
 - /config/configs.yaml and
 - /config/secrets.
- We wait for 1h to be able to inspect container

Note: we typically expect a program to not just print the secrets

<https://github.com/scontain/java-args-env-file.git>

```
public static void main(String[] args) {  
    // Print CLI arguments  
    System.out.println("CLI Arguments:");  
    for (int i = 0; i < args.length; i++) {  
        System.out.printf("arg[%d]: %s\n", i, args[i]);  
    }  
  
    // Print environment variables  
    System.out.println("\nEnvironment Variables:");  
    for (Map.Entry<String, String> env : System.getenv().entrySet()) {  
        System.out.printf("%s=%s\n", env.getKey(), env.getValue());  
    }  
  
    // Read and print /config/configs.yaml  
    readAndPrintFile("/config/configs.yaml", "Config File");  
  
    // Read and print /config/secrets  
    readAndPrintFile("/config/secrets", "Secrets File");  
  
    // Sleep for 1 hour  
    System.out.println("\n⌚ Sleeping for 1 hour to keep the container alive...");  
    try {  
        Thread.sleep(3600_000); // 1 hour = 3600000 ms  
    } catch (InterruptedException e) {  
        System.err.println("Sleep interrupted: " + e.getMessage());  
    }  
  
    System.out.println("✅ Done.");  
}
```

Hello World Program

The Java program:

- Prints CLI arguments.
- Prints environment variables.
- Prints the content of files
 - /config/configs.yaml and
 - /config/secrets.
- We wait for 1h to be able to inspect container

Note: we typically expect a program to not just print the secrets

<https://github.com/scontain/java-args-env-file.git>

```
public static void main(String[] args) {  
    // Print CLI arguments  
    System.out.println("CLI Arguments:");  
    for (int i = 0; i < args.length; i++) {  
        System.out.printf("arg[%d]: %s\n", i, args[i]);  
    }  
  
    // Print environment variables  
    System.out.println("\nEnvironment Variables:");  
    for (Map.Entry<String, String> env : System.getenv().entrySet()) {  
        System.out.printf("%s=%s\n", env.getKey(), env.getValue());  
    }  
  
    // Read and print /config/configs.yaml  
    readAndPrintFile("/config/configs.yaml", "Config File");  
  
    // Read and print /config/secrets  
    readAndPrintFile("/config/secrets", "Secrets File");  
  
    // Sleep for 1 hour  
    System.out.println("\n⌚ Sleeping for 1 hour to keep the container alive...");  
    try {  
        Thread.sleep(3600_000); // 1 hour = 3600000 ms  
    } catch (InterruptedException e) {  
        System.err.println("Sleep interrupted: " + e.getMessage());  
    }  
  
    System.out.println("✅ Done.");  
}
```


Hello World Program

The Java program:

- Prints CLI arguments.
- Prints environment variables.
- Prints the content of files
 - /config/configs.yaml and
 - /config/secrets.
- We wait for 1h to be able to inspect container

Note: we typically expect a program to not just print the secrets

<https://github.com/scontain/java-args-env-file.git>

```
public static void main(String[] args) {  
    // Print CLI arguments  
    System.out.println("CLI Arguments:");  
    for (int i = 0; i < args.length; i++) {  
        System.out.printf("arg[%d]: %s\n", i, args[i]);  
    }  
  
    // Print environment variables  
    System.out.println("\nEnvironment Variables:");  
    for (Map.Entry<String, String> env : System.getenv().entrySet()) {  
        System.out.printf("%s=%s\n", env.getKey(), env.getValue());  
    }  
  
    // Read and print /config/configs.yaml  
    readAndPrintFile("/config/configs.yaml", "Config File");  
  
    // Read and print /config/secrets  
    readAndPrintFile("/config/secrets", "Secrets File");  
  
    // Sleep for 1 hour  
    System.out.println("\n⌚ Sleeping for 1 hour to keep the container alive...");  
    try {  
        Thread.sleep(3600_000); // 1 hour = 3600000 ms  
    } catch (InterruptedException e) {  
        System.err.println("Sleep interrupted: " + e.getMessage());  
    }  
  
    System.out.println("✅ Done.");  
}
```

Hello World Program

The Java program:

- Prints CLI arguments.
- Prints environment variables.
- Prints the content of files
 - /config/configs.yaml and
 - /config/secrets.
- We wait for 1h to be able to inspect container

Note: we typically expect a program to not just print the secrets

<https://github.com/scontain/java-args-env-file.git>

```
public static void main(String[] args) {  
    // Print CLI arguments  
    System.out.println("CLI Arguments:");  
    for (int i = 0; i < args.length; i++) {  
        System.out.printf("arg[%d]: %s\n", i, args[i]);  
    }  
  
    // Print environment variables  
    System.out.println("\nEnvironment Variables:");  
    for (Map.Entry<String, String> env : System.getenv().entrySet()) {  
        System.out.printf("%s=%s\n", env.getKey(), env.getValue());  
    }  
  
    // Read and print /config/configs.yaml  
    readAndPrintFile("/config/configs.yaml", "Config File");  
  
    // Read and print /config/secrets  
    readAndPrintFile("/config/secrets", "Secrets File");  
  
    // Sleep for 1 hour  
    System.out.println("\n⌚ Sleeping for 1 hour to keep the container alive...");  
    try {  
        Thread.sleep(3600_000); // 1 hour = 3600000 ms  
    } catch (InterruptedException e) {  
        System.err.println("Sleep interrupted: " + e.getMessage());  
    }  
  
    System.out.println("✅ Done.");  
}
```


Hello World Program

The Java program:

- Prints CLI arguments.
- Prints environment variables.
- Prints the content of files
 - /config/configs.yaml and
 - /config/secrets.
- We wait for 1h to be able to inspect container

Note: we typically expect a program to not just print the secrets

<https://github.com/scontain/java-args-env-file.git>

```
public static void main(String[] args) {  
    // Print CLI arguments  
    System.out.println("CLI Arguments:");  
    for (int i = 0; i < args.length; i++) {  
        System.out.printf("arg[%d]: %s\n", i, args[i]);  
    }  
  
    // Print environment variables  
    System.out.println("\nEnvironment Variables:");  
    for (Map.Entry<String, String> env : System.getenv().entrySet()) {  
        System.out.printf("%s=%s\n", env.getKey(), env.getValue());  
    }  
  
    // Read and print /config/configs.yaml  
    readAndPrintFile("/config/configs.yaml", "Config File");  
  
    // Read and print /config/secrets  
    readAndPrintFile("/config/secrets", "Secrets File");  
  
    // Sleep for 1 hour  
    System.out.println("\n⌚ Sleeping for 1 hour to keep the container alive...");  
    try {  
        Thread.sleep(3600_000); // 1 hour = 3600000 ms  
    } catch (InterruptedException e) {  
        System.err.println("Sleep interrupted: " + e.getMessage());  
    }  
  
    System.out.println("✅ Done.");  
}
```

Hello World Program

The Java program:

- Prints CLI arguments.
- Prints environment variables.
- Prints the content of files
 - /config/configs.yaml and
 - /config/secrets.
- We wait for 1h to be able to inspect container

Note: we typically expect a program to not just print the secrets

<https://github.com/scontain/java-args-env-file.git>

```
public static void main(String[] args) {
    // Print CLI arguments
    System.out.println("CLI Arguments:");
    for (int i = 0; i < args.length; i++) {
        System.out.printf("arg[%d]: %s\n", i, args[i]);
    }

    // Print environment variables
    System.out.println("\nEnvironment Variables:");
    for (Map.Entry<String, String> env : System.getenv().entrySet()) {
        System.out.printf("%s=%s\n", env.getKey(), env.getValue());
    }

    // Read and print /config/configs.yaml
    readAndPrintFile("/config/configs.yaml", "Config File");

    // Read and print /config/secrets
    readAndPrintFile("/config/secrets", "Secrets File");

    // Sleep for 1 hour
    System.out.println("\n⌚ Sleeping for 1 hour to keep the container alive...");
    try {
        Thread.sleep(3600_000); // 1 hour = 3600000 ms
    } catch (InterruptedException e) {
        System.err.println("Sleep interrupted: " + e.getMessage());
    }

    System.out.println("✅ Done.");
}
```

Running the program locally

Note:

- app is “fault-tolerant”: we do not need to provide “injected” configuration files

```
ubuntu@apple-1:~/GIT/java-args-env-file$ java Main.java
```

CLI Arguments:

Environment Variables:

PATH=/home/ubuntu/.local/bin:/home/ubuntu/go/bin:/usr/local/go/bin:/home/ubuntu/.cargo/bin:/usr/

local/:*.xbm=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=01;35:*.mng=01;35:

...

Config File (/config/configs.yaml):

Failed to read /config/configs.yaml: /config/configs.yaml

Secrets File (/config/secrets):

Failed to read /config/secrets: /config/secrets



Sleeping for 1 hour to keep the container alive...

Running the program locally

Note:

- app is “fault-tolerant”: we do not need to provide “injected” configuration files

```
ubuntu@apple-1:~/GIT/java-args-env-file$ java Main.java
```

```
CLI Arguments:
```

```
Environment Variables:
```

```
PATH=/home/ubuntu/.local/bin:/home/ubuntu/go/bin:/usr/local/go/bin:/home/ubuntu/.cargo/bin:/usr/
```

```
local/:*.xbm=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=01;35:*.mng=01;35:
```

```
...
```

```
Config File (/config/configs.yaml):
```

```
Failed to read /config/configs.yaml: /config/configs.yaml
```

```
Secrets File (/config/secrets):
```

```
Failed to read /config/secrets: /config/secrets
```



Sleeping for 1 hour to keep the container alive...

Deploying in Kubernetes cluster

Running on Kubernetes cluster

- brand new Granite Rapids CPU servers
- running on 3 VMs

```
ubuntu@apple-1:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	Ready	control-plane	5d20h	v1.33.2
k8s-worker-1	Ready	<none>	5d19h	v1.33.2
k8s-worker-2	Ready	<none>	5d19h	v1.33.2

Deploying in Kubernetes cluster

Running on Kubernetes cluster

- brand new Granite Rapids CPU servers
- running on 3 VMs

```
ubuntu@apple-1:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
k8s-master	Ready	control-plane	5d20h	v1.33.2
k8s-worker-1	Ready	<none>	5d19h	v1.33.2
k8s-worker-2	Ready	<none>	5d19h	v1.33.2

Manifests

We use three kind of manifests

- **secrets.yaml**: defines secrets that will later be mapped in Java program
- **configmap**: defines configuration file that is mapped into
- **deployment**: defines how to start the program and how to map secrets & configmap into the container

Note: we also support **job** which is created but not deployed.

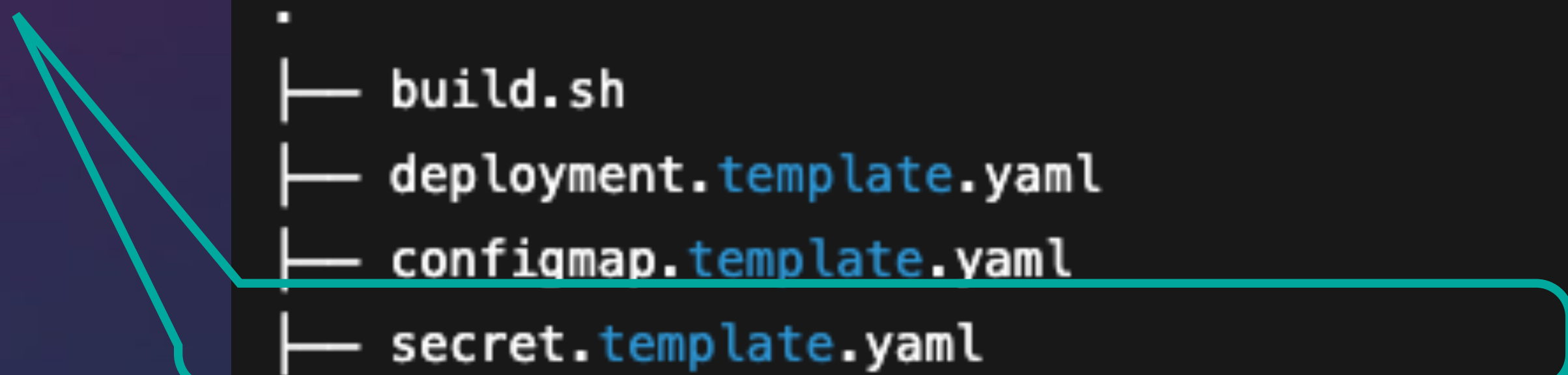
```
.
├── build.sh
├── deployment.template.yaml
├── configmap.template.yaml
├── secret.template.yaml
├── deployment.yaml           # <- generated
├── configmap.yaml           # <- generated
└── secret.yaml              # <- generated
```


Manifests

We use three kind of manifests

- **secrets.yaml**: defines secrets that will later be mapped in Java program
- **configmap**: defines configuration file that is mapped into
- **deployment**: defines how to start the program and how to map secrets & configmap into the container

Note: we also support **job** which is created but not deployed.



```
.
├── build.sh
├── deployment.template.yaml
├── configmap.template.yaml
├── secret.template.yaml
├── deployment.yaml      # ← generated
├── configmap.yaml       # ← generated
└── secret.yaml          # ← generated
```


Manifests

We use three kind of manifests

- **secrets.yaml**: defines secrets that will later be mapped in Java program
- **configmap**: defines configuration file that is mapped into
- **deployment**: defines how to start the program and how to map secrets & configmap into the container

Note: we also support **job** which is created but not deployed.

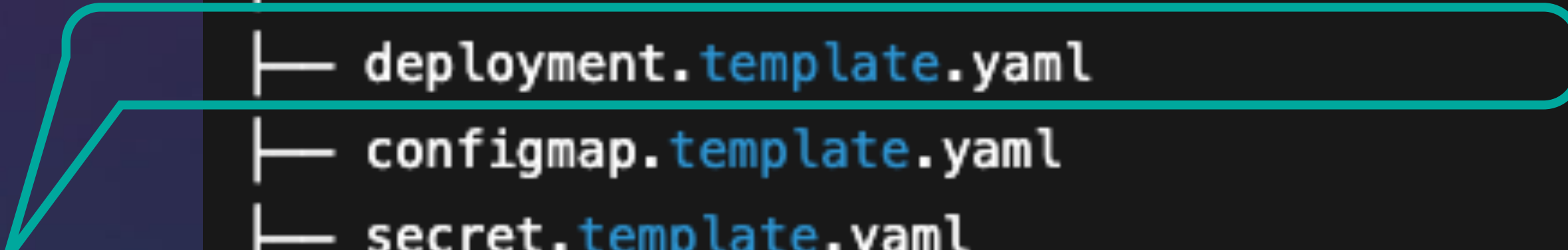
```
.
├── build.sh
├── deployment.template.yaml
├── configmap.template.yaml
├── secret.template.yaml
├── deployment.yaml      # <- generated
├── configmap.yaml      # <- generated
└── secret.yaml         # <- generated
```


Manifests

We use three kind of manifests

- **secrets.yaml**: defines secrets that will later be mapped in Java program
- **configmap**: defines configuration file that is mapped into
- **deployment**: defines how to start the program and how to map secrets & configmap into the container

Note: we also support **job** which is created but not deployed.



```
.
├── build.sh
├── deployment.template.yaml
├── configmap.template.yaml
├── secret.template.yaml
├── deployment.yaml      # <- generated
├── configmap.yaml       # <- generated
└── secret.yaml          # <- generated
```

Secret Template Manifest

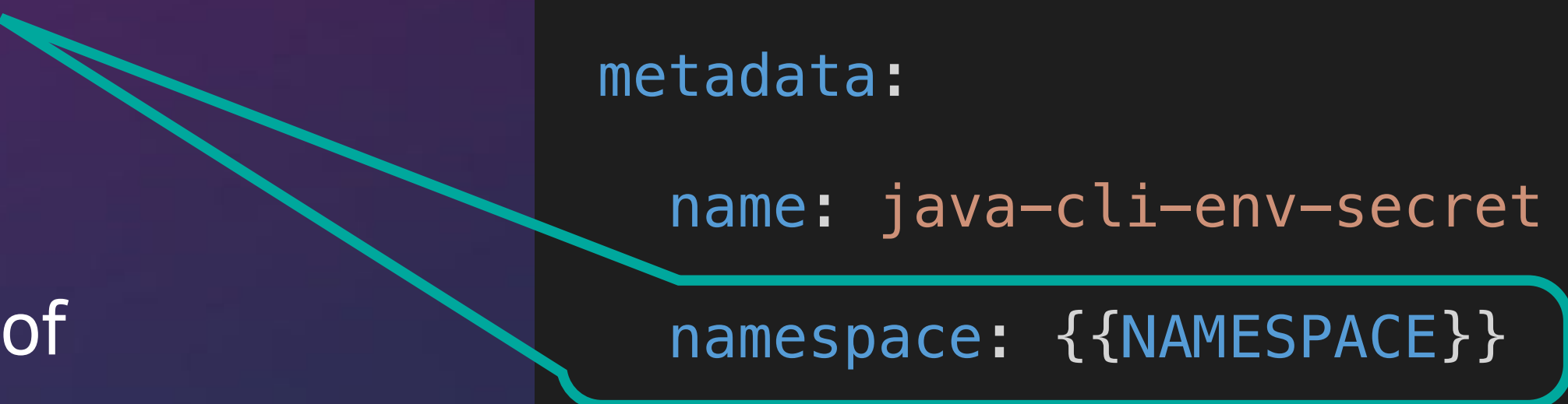
- We customize the manifests with
 - `{{HANDLEBAR}}`
- Our `build.sh` script replaces
 - the `{{HANDLEBAR}}` by value of
 - e.g., environment variable `$HANDLEBAR`
 - `HANDLEBAR` can be set via flags to the build script
- We want the secrets to stay secret
 - i.e., not visible in K8s cluster

```
apiVersion: v1
kind: Secret
metadata:
  name: java-cli-env-secret
  namespace: {{NAMESPACE}}
type: Opaque
stringData:
  secrets: |
    password: s3cr3t
    my_db_encryption_key: top_secret_key
```


Secret Template Manifest

- We customize the manifests with
 - `{{HANDLEBAR}}`
- Our `build.sh` script replaces
 - the `{{HANDLEBAR}}` by value of
 - e.g., environment variable `$HANDLEBAR`
 - `HANDLEBAR` can be set via flags to the build script
- We want the secrets to stay secret
 - i.e., not visible in K8s cluster

```
apiVersion: v1
kind: Secret
metadata:
  name: java-cli-env-secret
  namespace: {{NAMESPACE}}
type: Opaque
stringData:
  secrets: |
    password: s3cr3t
    my_db_encryption_key: top_secret_key
```



Secret Template Manifest

- We customize the manifests with
 - `{{HANDLEBAR}}`
- Our `build.sh` script replaces
 - the `{{HANDLEBAR}}` by value of
 - e.g., environment variable `$HANDLEBAR`
 - `HANDLEBAR` can be set via flags to the build script
- We want the secrets to stay secret
 - i.e., not visible in K8s cluster

```
apiVersion: v1
kind: Secret
metadata:
  name: java-cli-env-secret
  namespace: {{NAMESPACE}}
type: Opaque
stringData:
  secrets: |
    password: s3cr3t
    my_db_encryption_key: top_secret_key
```


ConfigMap Template Manifest

- We can customize the manifests with
 - `{{NAMESPACE}}`
- Dummy key value pairs in “`configs.yaml`”
 - `key1`, and
 - `key2`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: java-cli-env-config
  namespace: {{NAMESPACE}}
data:
  configs.yaml: |
    key1: value1
    key2: value2
```

ConfigMap Template Manifest

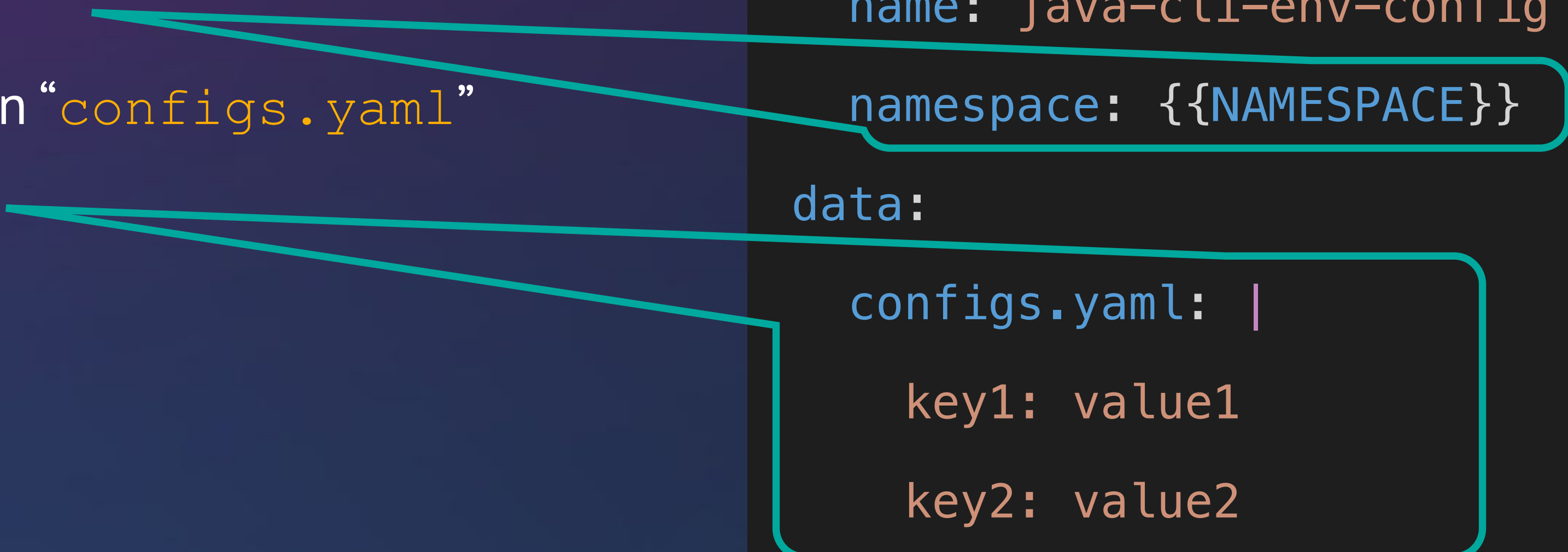
- We can customize the manifests with
 - `{{NAMESPACE}}`
- Dummy key value pairs in “`configs.yaml`”
 - `key1`, and
 - `key2`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: java-cli-env-config
  namespace: {{NAMESPACE}}
data:
  configs.yaml: |
    key1: value1
    key2: value2
```


ConfigMap Template Manifest

- We can customize the manifests with
 - `{{NAMESPACE}}`
- Dummy key value pairs in “`configs.yaml`”
 - `key1`, and
 - `key2`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: java-cli-env-config
  namespace: {{NAMESPACE}}
data:
  configs.yaml: |
    key1: value1
    key2: value2
```



Kubernetes Deployment

- Deployment has a
 - name: `java-cli-env-reader`
 - K8s namespace: `{{NAMESPACE}}`
- We require one replica:
 - identified by label: `java-cli-env-reader`
- Templates define how to create containers
 - they all have label: `java-cli-env-reader`
 - spec section defines each container of a pod

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-cli-env-reader
  namespace: {{NAMESPACE}}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: java-cli-env-reader
  template:
    metadata:
      labels:
        app: java-cli-env-reader
    spec:
      containers:
        - name: java-cli-env
```


Kubernetes Deployment

- Deployment has a
 - name: `java-cli-env-reader`
 - K8s namespace: `{{NAMESPACE}}`
- We require one replica:
 - identified by label: `java-cli-env-reader`
- Templates define how to create containers
 - they all have label: `java-cli-env-reader`
 - spec section defines each container of a pod

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: java-cli-env-reader
```

```
  namespace: {{NAMESPACE}}
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: java-cli-env-reader
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: java-cli-env-reader
```

```
    spec:
```

```
      containers:
```

```
        - name: java-cli-env
```

Kubernetes Deployment

- Deployment has a
 - name: `java-cli-env-reader`
 - K8s namespace: `{{NAMESPACE}}`
- We require one replica:
 - identified by label: `java-cli-env-reader`
- Templates define how to create containers
 - they all have label: `java-cli-env-reader`
 - spec section defines each container of a pod

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: java-cli-env-reader
```

```
  namespace: {{NAMESPACE}}
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: java-cli-env-reader
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: java-cli-env-reader
```

```
    spec:
```

```
      containers:
```

```
        - name: java-cli-env
```


Kubernetes Deployment

- Deployment has a
 - name: `java-cli-env-reader`
 - K8s namespace: `{{NAMESPACE}}`
- We require one replica:
 - identified by label: `java-cli-env-reader`
- Templates define how to create containers
 - they all have label: `java-cli-env-reader`
 - spec section defines each container of a pod

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-cli-env-reader
  namespace: {{NAMESPACE}}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: java-cli-env-reader
  template:
    metadata:
      labels:
        app: java-cli-env-reader
    spec:
      containers:
        - name: java-cli-env
```

Kubernetes Deployment

- Deployment has a
 - name: `java-cli-env-reader`
 - K8s namespace: `{{NAMESPACE}}`
- We require one replica:
 - identified by label: `java-cli-env-reader`
- Templates define how to create containers
 - they all have label: `java-cli-env-reader`
 - spec section defines each container of a pod

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: java-cli-env-reader
```

```
  namespace: {{NAMESPACE}}
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: java-cli-env-reader
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: java-cli-env-reader
```

```
    spec:
```

```
      containers:
```

```
        - name: java-cli-env
```


Kubernetes Deployment

- Deployment has a
 - name: `java-cli-env-reader`
 - K8s namespace: `{{NAMESPACE}}`
- We require one replica:
 - identified by label: `java-cli-env-reader`
- Templates define how to create containers
 - they all have label: `java-cli-env-reader`
 - spec section defines each container of a pod

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-cli-env-reader
  namespace: {{NAMESPACE}}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: java-cli-env-reader
  template:
    metadata:
      labels:
        app: java-cli-env-reader
    spec:
      containers:
        - name: java-cli-env
```

Kubernetes Deployment

- Deployment has a
 - name: `java-cli-env-reader`
 - K8s namespace: `{{NAMESPACE}}`
- We require one replica:
 - identified by label: `java-cli-env-reader`
- Templates define how to create containers
 - they all have label: `java-cli-env-reader`
 - spec section defines each container of a pod

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: java-cli-env-reader
  namespace: {{NAMESPACE}}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: java-cli-env-reader
  template:
    metadata:
      labels:
        app: java-cli-env-reader
    spec:
      containers:
        - name: java-cli-env
```

Kubernetes Deployment

- Container spec
 - container name: `java-cli-env`
 - we pass some args: `arg1, arg2`
 - image name: `Repository : Tag`
 - on each (re-)start K8s checks if image was updated: `always`
 - NOTE: container will not even if the content of the container would be updated

```
spec:
  replicas: 1
  selector:
    matchLabels:
      app: java-cli-env-reader
  template:
    metadata:
      labels:
        app: java-cli-env-reader
    spec:
      containers:
        - name: java-cli-env
          image: {{REPO}}:{{TAG}}
          imagePullPolicy: Always
          args: ["arg1", "arg2"]
          volumeMounts:
            - name: config-volume
```


Kubernetes Deployment

- Container spec
 - container name: `java-cli-env`
 - we pass some args: `arg1, arg2`
 - image name: `Repository : Tag`
 - on each (re-)start K8s checks if image was updated: `always`
 - NOTE: container will not even if the content of the container would be updated

```
spec:  
  replicas: 1  
  selector:  
    matchLabels:  
      app: java-cli-env-reader  
  template:  
    metadata:  
      labels:  
        app: java-cli-env-reader  
    spec:  
      containers:  
        - name: java-cli-env  
          image: {{REPO}}:{{TAG}}  
          imagePullPolicy: Always  
          args: ["arg1", "arg2"]  
          volumeMounts:  
            - name: config-volume
```

Kubernetes Deployment

- Container spec
 - container name: `java-cli-env`
 - we pass some args: `arg1, arg2`
 - image name: `Repository : Tag`
 - on each (re-)start K8s checks if image was updated: `always`
 - NOTE: container will not even if the content of the container would be updated

```
spec:  
  replicas: 1  
  selector:  
    matchLabels:  
      app: java-cli-env-reader  
  template:  
    metadata:  
      labels:  
        app: java-cli-env-reader  
    spec:  
      containers:  
        - name: java-cli-env  
          image: {{REPO}}:{{TAG}}  
          imagePullPolicy: Always  
          args: ["arg1", "arg2"]  
          volumeMounts:  
            - name: config-volume
```

Kubernetes Deployment

- Container spec
 - container name: `java-cli-env`
 - we pass some args: `arg1, arg2`
 - image name: `Repository : Tag`
 - on each (re-)start K8s checks if image was updated: `always`
 - NOTE: container will not even if the content of the container would be updated

```
spec:  
  replicas: 1  
  selector:  
    matchLabels:  
      app: java-cli-env-reader  
  template:  
    metadata:  
      labels:  
        app: java-cli-env-reader  
    spec:  
      containers:  
        - name: java-cli-env  
          image: {{REPO}}:{{TAG}}  
          imagePullPolicy: Always  
          args: ["arg1", "arg2"]  
          volumeMounts:  
            - name: config-volume
```


Kubernetes Deployment

- Container spec
 - container name: `java-cli-env`
 - we pass some args: `arg1, arg2`
 - image name: `Repository : Tag`
 - on each (re-)start K8s checks if image was updated: `always`
 - NOTE: container will not even if the content of the container would be updated

```
spec:  
  replicas: 1  
  selector:  
    matchLabels:  
      app: java-cli-env-reader  
  template:  
    metadata:  
      labels:  
        app: java-cli-env-reader  
    spec:  
      containers:  
        - name: java-cli-env  
          image: {{REPO}}:{{TAG}}  
          imagePullPolicy: Always  
          args: ["arg1", "arg2"]  
          volumeMounts:  
            - name: config-volume
```

volumeMounts

- VolumeMount
 - config-volume
 - secret-volume
- We mount into the container
 - the config map: /config/configs.yaml
 - the secrets: /config/secrets

```
image: {{REPO}}:{{TAG}}
```

```
imagePullPolicy: Always
```

```
args: ["arg1", "arg2"]
```

```
volumeMounts:
```

```
  - name: config-volume
```

```
    mountPath: /config/configs.yaml
```

```
    subPath: configs.yaml
```

```
  - name: secret-volume
```

```
    mountPath: /config/secrets
```

```
    subPath: secrets
```

```
imagePullSecrets:
```

```
  - name: {{PULLSECRET}}
```

```
volumes:
```

```
  - name: config-volume
```

```
    configMap:
```

```
      name: java-cli-env-config
```

volumeMounts

- VolumeMount
 - config-volume
 - secret-volume
- We mount into the container
 - the config map: /config/configs.yaml
 - the secrets: /config/secrets

```
image: {{REPO}}:{{TAG}}
```

```
imagePullPolicy: Always
```

```
args: ["arg1", "arg2"]
```

```
volumeMounts:
```

```
– name: config-volume
```

```
  mountPath: /config/configs.yaml
```

```
  subPath: configs.yaml
```

```
– name: secret-volume
```

```
  mountPath: /config/secrets
```

```
  subPath: secrets
```

```
imagePullSecrets:
```

```
– name: {{PULLSECRET}}
```

```
volumes:
```

```
– name: config-volume
```

```
  configMap:
```

```
    name: java-cli-env-config
```


volumeMounts

- VolumeMount
 - config-volume
 - secret-volume
- We mount into the container
 - the config map: /config/configs.yaml
 - the secrets: /config/secrets

```
image: {{REPO}}:{{TAG}}
```

```
imagePullPolicy: Always
```

```
args: ["arg1", "arg2"]
```

```
volumeMounts:
```

```
  - name: config-volume
```

```
    mountPath: /config/configs.yaml
```

```
    subPath: configs.yaml
```

```
  - name: secret-volume
```

```
    mountPath: /config/secrets
```

```
    subPath: secrets
```

```
imagePullSecrets:
```

```
  - name: {{PULLSECRET}}
```

```
volumes:
```

```
  - name: config-volume
```

```
    configMap:
```

```
      name: java-cli-env-config
```

Volumes

- We define the volumes:
 - `config-volume`
 - `secret-volume`
- with the help of
 - `configMap : java-cli-env-config`
 - `secret: java-cli-env-secret`

```
    mountPath: /config/configs.yaml
    subPath: configs.yaml
  - name: secret-volume
    mountPath: /config/secrets
    subPath: secrets
imagePullSecrets:
  - name: {{PULLSECRET}}
volumes:
  - name: config-volume
    configMap:
      name: java-cli-env-config
  - name: secret-volume
    secret:
      secretName: java-cli-env-secret
```

Volumes

- We define the volumes:
 - config-volume
 - secret-volume
- with the help of
 - configMap : java-cli-env-config
 - secret: java-cli-env-secret

```
    mountPath: /config/configs.yaml
    subPath: configs.yaml
  - name: secret-volume
    mountPath: /config/secrets
    subPath: secrets
  imagePullSecrets:
    - name: {{PULLSECRET}}
  volumes:
    - name: config-volume
      configMap:
        name: java-cli-env-config
    - name: secret-volume
      secret:
        secretName: java-cli-env-secret
```


Volumes

- We define the volumes:
 - `config-volume`
 - `secret-volume`
- with the help of
 - `configMap : java-cli-env-config`
 - `secret: java-cli-env-secret`

```
    mountPath: /config/configs.yaml
    subPath: configs.yaml
  - name: secret-volume
    mountPath: /config/secrets
    subPath: secrets
imagePullSecrets:
  - name: {{PULLSECRET}}
volumes:
  - name: config-volume
    configMap:
      name: java-cli-env-config
  - name: secret-volume
    secret:
      secretName: java-cli-env-secret
```

Build and Deploy

Execute:

- `./scripts/build.sh` - create image and manifests from templates
- `./scripts/deploy.sh` - deploy manifests to cluster

```
ubuntu@apple-1:java-args-env-file$ ./scripts/build.sh
```

```
...
```

```
 Generating Kubernetes manifests
```

```
 Creating configmap.yaml
```

```
 Creating deployment.yaml
```

```
 Creating job.yaml
```

```
 Creating secret.yaml
```

```
 All manifests generated
```

```
ubuntu@apple-1:java-args-env-file$ ./scripts/deploy.sh
```

```
 Deploying resources using context: <current>
```

```
configmap/java-cli-env-config created
```

```
secret/java-cli-env-secret created
```

```
deployment.apps/java-cli-env-reader created
```

```
 Deployment complete.
```